

Полная документация для ДЭМО

09.02.07 2026

Оглавление

1. Используемые библиотеки и ассет.....	4
Python (бэкенд)	4
Фронтенд (ассеты, подключаются локально)	4
2. Настройка окружения (шаг за шагом).....	5
2.1. Установка Python (3.10 или новее)	5
2.2. Установка PostgreSQL	5
2.3. Подготовка папки проекта	5
2.4. Установка зависимостей Python	6
2.5. Настройка статика и шрифтов	6
2.6. Запуск приложения	6
3. Файловая структура (итоговая)	7
4. Пример кода для всех файлов	8
4.1. main.py	8
4.2. static/style.css	12
4.3. Шаблоны (папка templates/)	17
register.html	17
login.html	18
profile.html	18
create_request.html	20
admin.html	21
5. Команды и полезные инструкции.....	22
5.1. Установка PostgreSQL и создание БД (если на экзамене чистая система)	22
5.2. Установка Python-зависимостей	22
5.3. Запуск сервера	22
6. Полезные фишки (готовые блоки кода для копирования)	23
6.1. Слайдер на Glide.js (профиль или любая страница)	23
6.1.1. Подключение библиотек (уже в шапке профиля).....	23
6.1.2. HTML-разметка слайдера (вставить в нужное место).....	23
6.1.3. Инициализация слайдера.....	23
6.1.4. Минимальные стили.....	23
6.2. Админка: пагинация, сортировка, всплывающие уведомления.....	24
6.2.1. Бэкенд (фрагмент main.py)	24
6.2.2. Фронтенд (admin.html) – фрагменты.....	26
6.2.3. CSS (добавить в style.css).....	27
6.3. Динамическое поле «Сдача с» (готовый код).....	27

6.3.1. Бэкенд – добавить колонку в БД (один раз выполнить в PostgreSQL).....	27
6.3.2. Обновлённый post_create_request.....	27
6.3.3. Фронтенд (create_request.html) – добавки в форму	28
6.4 Полный css (Более красивый)	28

1. Используемые библиотеки и ассет

Python (бэкенд)

Библиотека	Версия (актуальная)	Назначение
fastapi	0.115+	Веб-фреймворк
uvicorn	0.30+	ASGI-сервер для запуска
psycopg2-binary	2.9+	Драйвер PostgreSQL
python-multipart	0.0.9+	Обработка форм (Form(...))
bcrypt (опционально)	4.0+	Хеширование паролей (рекомендуется)

Фронтенд (ассеты, подключаются локально)

Файл	Источник	Назначение
glide.core.min.css	Glide.js (v3.7.1)	Базовые стили слайдера
glide.theme.min.css	Glide.js	Тематические стили (кнопки, пагинация)
glide.min.js	Glide.js	Логика слайдера
PT Sans шрифт	ParaType (Open Font License)	Основной шрифт (regular, bold, italic)
style.css	Самописный	Кастомные стили приложения

2. Настройка окружения (шаг за шагом)

2.1. Установка Python (3.10 или новее)

- Скачай установщик с официального сайта
- При установке отметь **Add Python to PATH**.

2.2. Установка PostgreSQL

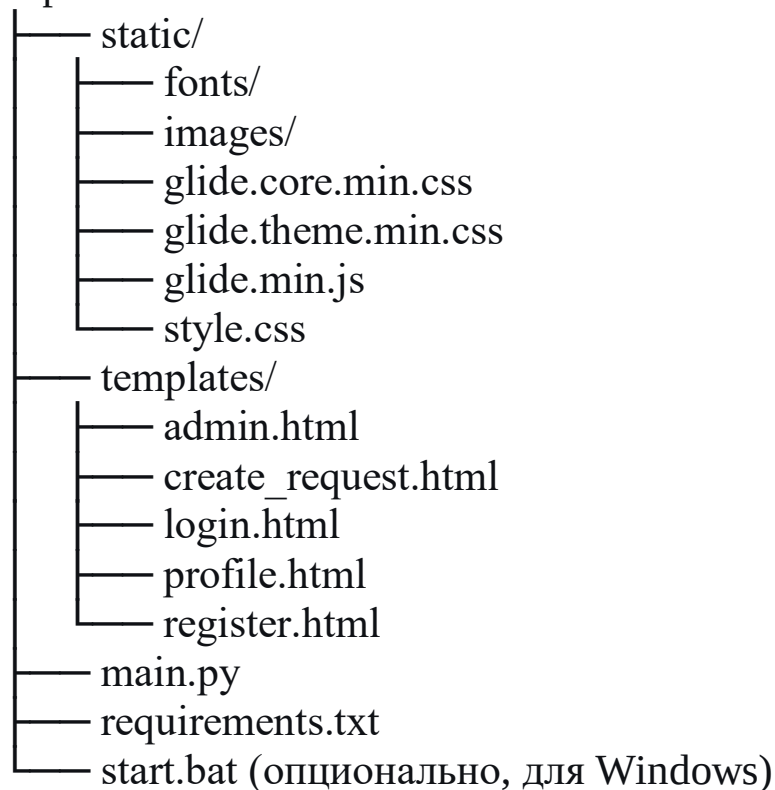
- Скачай установщик с [postgresql.org](https://www.postgresql.org) (например, версию 16).
- При установке запомни пароль суперпользователя (postgres).
- После установки запусти **pgAdmin** или **psql** и создай базу данных:
CREATE DATABASE passRF;

Если имя базы отличается, измени строку подключения в `main.py`.

2.3. Подготовка папки проекта

Создай папку. Внутри создай следующие подпапки:

Папка/



2.4. Установка зависимостей Python

Создай файл requirements.txt:

```
fastapi
uvicorn
jinja2
python-multipart
psycopg2-binary
bcrypt
```

Затем в терминале (из папки проекта):

```
pip install -r requirements.txt
```

Если нет интернета, заранее скачайте .whl файлы и установите вручную, либо используйте заранее подготовленный venv с копированием (см. инструкцию по переносу).

2.5. Настройка статики и шрифтов

- Скопируй файлы glide.* в папку static/.
- Шрифты PT Sans (.ttf) положи в static/fonts/.
- Четыре картинки для слайдера положи в static/images/.
- Файл style.css положи в static/.

2.6. Запуск приложения

```
uvicorn main:app --reload
```

При надобности можно добавить порт --port 8000

(Если python зависнет, то смена порта может помочь))

После запуска открывай в браузере: http://localhost:8000

3. Файловая структура (итоговая)

```
Папка/
|
├─ main.py                # Основной код приложения
├─ requirements.txt       # Зависимости Python
|
├─ static/
|   └─ fonts/
|       ├── PTSans-Regular.ttf
|       ├── PTSans-Bold.ttf
|       └─ PTSans-Italic.ttf
|   └─ images/
|       ├── image (1).jpeg
|       ├── image (2).jpeg
|       ├── image (3).jpeg
|       └─ image (4).jpeg
|   └─ glide.core.min.css
|   └─ glide.theme.min.css
|   └─ glide.min.js
|   └─ style.css
|
└─ templates/
    ├── admin.html
    ├── create_request.html
    ├── login.html
    ├── profile.html
    └─ register.html
```

4. Пример кода для всех файлов

4.1. main.py

```
import re
import psycopg2
import psycopg2.extras
from fastapi import FastAPI, Query, Form, Request
from fastapi.templating import Jinja2Templates
from fastapi.responses import RedirectResponse
from fastapi.staticfiles import StaticFiles
from starlette.middleware.sessions import SessionMiddleware

app = FastAPI()
app.add_middleware(SessionMiddleware, secret_key="Banana")
app.mount("/static", StaticFiles(directory="static"))
templates = Jinja2Templates(directory="templates")

def get_db():
    conn = psycopg2.connect(database="passRF", user="postgres", password="123", host=
"localhost", port="5432")
    conn.cursor_factory = psycopg2.extras.DictCursor
    return conn

def init_db():
    with get_db() as conn:
        cur = conn.cursor()
        cur.execute("""
            create table if not exists users(
                id serial primary key,
                login text unique not null,
                password text not null,
                date_b text not null,
                fio text not null,
                email text not null,
                phone text not null
            );
        """)
        cur.execute("""
            create table if not exists requests(
                id serial primary key,
                user_id integer references users(id) on delete cascade,
                transport text not null,
                date_start text not null,
                payment_method text not null,
```



```

        status text default 'Новая',
        review text
    );
    """
    conn.commit()

init_db()

@app.get("/")
def get_base(request: Request):
    return RedirectResponse("/login", status_code=302)

@app.get("/logout")
def get_logout(request: Request):
    request.session.clear()
    return RedirectResponse("/login", status_code=302)

@app.get("/register")
def get_register(request: Request):
    return templates.TemplateResponse("register.html", {"request": request, "error":
None})

@app.get("/login")
def get_login(request: Request):
    return templates.TemplateResponse("login.html", {"request": request, "error": Non
e})

@app.post("/register")
def post_register(request: Request,
    login: str = Form(...),
    password: str = Form(...),
    date_b: str = Form(...),
    fio: str = Form(...),
    email: str = Form(...),
    phone: str = Form(...)):
    error = None
    if not re.fullmatch(r'[a-zA-Z0-9]{6,}', login):
        error = "Логин: латиница и цифры, минимум 6 символов"
    elif len(password) < 8:
        error = "Пароль: минимум 8 символов"
    elif not re.fullmatch(r'[a-яА-ЯёЁ\s]+', fio):
        error = "ФИО: кириллица и пробелы"
    elif not re.fullmatch(r'^@+@[^@]+\.[^@]+', email):
        error = "Почта: неверный формат"

```

```

elif not re.fullmatch(r'8\(\d{3}\)\d{3}-\d{2}-\d{2}', phone):
    error = "Телефон: маска 8(XXX)XXX-XX-XX"
else:
    with get_db() as conn:
        cur = conn.cursor()
        cur.execute("select * from users where login = %s", (login,))
        user = cur.fetchone()
        if not user:
            cur.execute("insert into users (login,password,date_b,fio,email,phone
) values (%s,%s,%s,%s,%s,%s)",
                        (login, password, date_b, fio, email, phone))
            conn.commit()
            return RedirectResponse("/login", status_code=302)
        else:
            error = "Логин уже занят"
    return templates.TemplateResponse("register.html", {"request": request, "error":
error,
                                                    "login": login, "date_b": dat
e_b, "fio": fio,
                                                    "email": email, "phone": phon
e})

@app.post("/login")
def post_login(request: Request, login: str = Form(...), password: str = Form(...)):
    if login == "Admin26":
        if password == "Demo20":
            request.session["admin"] = True
            return RedirectResponse("/admin", status_code=302)
        else:
            return templates.TemplateResponse("login.html", {"request": request, "err
or": "Неверные данные"})
    with get_db() as conn:
        cur = conn.cursor()
        cur.execute("select * from users where login = %s", (login,))
        user = cur.fetchone()
        if user and user["password"] == password:
            request.session["user_id"] = user["id"]
            return RedirectResponse("/profile", status_code=302)
    return templates.TemplateResponse("login.html", {"request": request, "error": "Не
верные данные"})

@app.get("/profile")
def get_profile(request: Request):
    user_id = request.session.get("user_id")

```

```

    if not user_id:
        return RedirectResponse("/login", status_code=302)
    with get_db() as conn:
        cur = conn.cursor()
        cur.execute("select * from requests where user_id = %s", (user_id,))
        requests_list = cur.fetchall()
    return templates.TemplateResponse("profile.html", {"request": request, "requests"
: requests_list})

TRANSPORTS = ["Автобус", "Электробус", "Трамвай"]
PAYMENTS = ["Наличные", "Перевод по номеру"]

@app.get("/create_request")
def get_create_request(request: Request):
    user_id = request.session.get("user_id")
    if not user_id:
        return RedirectResponse("/login", status_code=302)
    return templates.TemplateResponse("create_request.html", {"request": request,
"transports": TRANSPORT
S,
"payments": PAYMENTS})

@app.post("/create_request")
def post_create_request(request: Request,
                        transport: str = Form(...),
                        date: str = Form(...),
                        payment: str = Form(...)):
    user_id = request.session.get("user_id")
    if not user_id:
        return RedirectResponse("/login", status_code=302)
    with get_db() as conn:
        cur = conn.cursor()
        cur.execute("insert into requests (transport,date_start,payment_method,user_i
d) values (%s,%s,%s,%s)",
                    (transport, date, payment, user_id))
        conn.commit()
    return RedirectResponse("/profile", status_code=302)

@app.get("/admin")
def get_admin(request: Request, status: str | None = Query(None)):
    if not request.session.get("admin"):
        return RedirectResponse("/login", status_code=302)
    base = "select requests.* , users.login, users.fio from requests, users where req
uests.user_id = users.id"

```

```

params = []
if status and status != 'Bce':
    base += " and status = %s"
    params.append(status)
with get_db() as conn:
    cur = conn.cursor()
    cur.execute(base, params)
    requests_list = cur.fetchall()
return templates.TemplateResponse("admin.html", {"request": request,
                                                    "requests": requests_list,
                                                    "cur_status": status or 'Bce'})

@app.post("/add_review")
def post_add_review(request: Request, request_id: str = Form(...), review: str = Form(
...)):
    user_id = request.session.get("user_id")
    if not user_id:
        return RedirectResponse("/login", status_code=302)
    with get_db() as conn:
        cur = conn.cursor()
        cur.execute("select * from requests where id = %s and status = 'Обучение заве
ршено' and review is null", (request_id,))
        isdate = cur.fetchone()
        if isdate:
            cur.execute("update requests set review = %s where id = %s", (review, req
uest_id))
            conn.commit()
        return RedirectResponse("/profile", status_code=302)

@app.post("/admin/change_status")
def post_change_status(request: Request, request_id: str = Form(...), status: str = F
orm(...)):
    if not request.session.get("admin"):
        return RedirectResponse("/login", status_code=302)
    with get_db() as conn:
        cur = conn.cursor()
        cur.execute("update requests set status = %s where id = %s", (status, request
_id))
        conn.commit()
    return RedirectResponse("/admin", status_code=302)

```

4.2. static/style.css

```
/* Regular */
@font-face {
  font-family: 'PT Sans';
  src: url('/static/fonts/PTSans-Regular.ttf') format('truetype');
  font-weight: 400;
  font-style: normal;
  font-display: swap;
}

/* Bold */
@font-face {
  font-family: 'PT Sans';
  src: url('/static/fonts/PTSans-Bold.ttf') format('truetype');
  font-weight: 700;
  font-style: normal;
  font-display: swap;
}

/* Italic */
@font-face {
  font-family: 'PT Sans';
  src: url('/static/fonts/PTSans-Italic.ttf') format('truetype');
  font-weight: 400;
  font-style: italic;
  font-display: swap;
}

body {
  font-family: 'PT Sans', sans-serif;
  background-color: whitesmoke;
  align-items: center;
  text-align: center;
  margin: 0;
  padding: 10px;
  display: flex;
  flex-direction: column;
  font-size: 16px;
}

h1 {
  font-size: 36px;
  color: #007bff;
}
```

```
h2 {
  font-size: 24px;
  color: #007bff;
}

form {
  background-color: white;
  padding: 15px;
  box-shadow: 1px 5px 5px 1px gray;
  border-radius: 15px;
  font-weight: bold;
  max-width: 500px;
}

.elebus{
  box-shadow: 1px 5px 5px 1px gray;
  border-radius: 15px;
  max-width: 500px;
  margin-bottom: 20px;
}

input, button, textarea, select{
  padding: 10px;
  margin: 10px 0;
  border-radius: 15px;
  width: 100%;
  max-width: 400px;
  box-sizing: border-box;
}

button {
  border: none;
  background-color: #0d47a1;
  transform: 0,3s;
  cursor: pointer;
  color: white;
  width: 60%;
}

button:hover{
  background-color: rgb(40, 40, 177);
}
```

```
a{
  color: #007bff;
  font-weight: bold;
}

.nav{
  padding-bottom: 20px;
}

.card{
  background-color: white;
  padding: 15px;
  margin: 10px;
  box-shadow: 1px 5px 5px 1px gray;
  border-radius: 15px;
  width: 100%;
  max-width: 400px;
}

.form-review, .form-status{
  box-shadow: none;
  padding: 0px;
}

.form-filter{
  box-shadow: none;
  background-color: whitesmoke;
  padding-bottom: 0px;
}

.form-status select, .form-status button {
  width: auto;
}

.table-div{
  width: 100%;
  overflow-x: auto;
  margin: 10px;
}

table{
  background-color: white;
  box-shadow: 1px 5px 5px 1px gray;
```

```
    width: 100%;
    border-collapse: collapse;
}

.table-responsive {
    overflow-x: auto;
    -webkit-overflow-scrolling: touch; /* плавный скролл на iOS */
    max-width: 100%;
}

th{
    background-color: #007bff;
    color: white;
    font-weight: bold;
}

th, td{
    border: 1px solid grey;
    padding: 10px;
}

.glide {
    max-width: 600px;
}

.glide__slide img {
    width: 100%;
    height: 100%;
    object-fit: cover;
    display: block;
}

.glide__arrow {
    width: 40px;
}

@media (max-width: 480px) {
    body {
        padding: 8px;
    }
    .card, form:not(.form-filter):not(.form-status):not(.form-review) {
        margin: 12px 0;
        padding: 16px;
    }
}
```



```

        button[type="submit"] {
            max-width: 100%;
        }
        th, td {
            padding: 8px 6px;
            font-size: 14px;
        }
        .glide {
            max-width: 100%;
        }
        .elebus{
            max-width: 100%;
        }
    }
}

```

4.3. Шаблоны (папка templates/)

register.html

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="/static/style.css">
    <title>Регистрация</title>
</head>
<body>
    <h1>Регистрация</h1>
    {% if error %}
        <p style="color: red;">{{ error }}</p>
    {% endif %}
    <form method="post">
        Логин: <input type="text" name="login" value="{{ login }}" required><br>
        Пароль: <input type="password" name="password" required><br>
        ФИО: <input type="text" name="fio" value="{{ fio }}" required><br>
        Дата рождения: <input type="date" name="date_b" value="{{ date_b }}" required
    ><br>
        Почта: <input type="email" name="email" value="{{ email }}" required><br>
        Телефон: <input type="text" name="phone" value="{{ phone }}" placeholder="8(X
XX)XXX-XX-XX" required><br>
        <button type="submit">Подтвердить</button>
    </form>
    <p>Уже зарегистрированы? <a href="/login">Вход</a></p>
</body>

```

```
</html>
```

login.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="/static/style.css">
  <title>Вход</title>
</head>
<body>
  <h1>Вход</h1>
  {% if error %}
    <p style="color: red;">{{ error }}</p>
  {% endif %}
  <form method="post">
    Логин: <input type="text" name="login" required><br>
    Пароль: <input type="password" name="password" required><br>
    <button type="submit">Подтвердить</button>
  </form>
  <p>Еще не зарегистрированы? <a href="/register">Регистрация</a></p>
</body>
</html>
```

profile.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="/static/style.css">
  <link rel="stylesheet" href="/static/glide.core.min.css">
  <link rel="stylesheet" href="/static/glide.theme.min.css">
  <title>Профиль</title>
</head>
<body>
  <h1>Профиль</h1>
  <div class="nav">
    <a href="/create_request">Новая заявка</a> | <a href="/logout">Выйти</a>
  </div>

  <!-- Слайдер -->
  <div class="glide">
```

```

<div class="glide__track" data-glide-el="track">
  <ul class="glide__slides">
    <li class="glide__slide"><
/li>
    <li class="glide__slide"><
/li>
    <li class="glide__slide"><
/li>
    <li class="glide__slide"><
/li>
  </ul>
</div>
<div class="glide__arrows" data-glide-el="controls">
  <button class="glide__arrow glide__arrow--left" data-glide-dir="<"></but
ton>
  <button class="glide__arrow glide__arrow--right" data-glide-dir=">"></bu
tton>
</div>
</div>

<h2>История заявок</h2>
{% for r in requests %}
<div class="card">
  <p><strong>Вид транспорта:</strong> {{ r.transport }}</p>
  <p><strong>Дата старта:</strong> {{ r.date_start }}</p>
  <p><strong>Способ оплаты:</strong> {{ r.payment_method }}</p>
  <p><strong>Статус:</strong> {{ r.status }}</p>
  {% if r.review %}
    <p><strong>Отзыв:</strong> {{ r.review }}</p>
  {% endif %}
  {% if r.review is none and r.status == 'Обучение завершено' %}
    <form method="post" action="/add_review" class="form-review">
      <input type="hidden" name="request_id" value="{{ r.id }}">
      <textarea name="review" placeholder="Отзыв" required></textarea>
      <button type="submit">Подтвердить</button>
    </form>
  {% endif %}
</div>
{% endfor %}

<script src="/static/glide.min.js"></script>
<script>
  new Glide('.glide', {
    type: 'carousel',

```

```

        autoplay: 3000,
        perView: 1,
        loop: true
    }).mount();
</script>
</body>
</html>

```

create_request.html

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="/static/style.css">
    <title>Новая заявка</title>
</head>
<body>
    <h1>Новая заявка</h1>
    <div class="nav">
        <a href="/profile">Назад</a>
    </div>
    {% if error %}
        <p style="color: red;">{{ error }}</p>
    {% endif %}
    <form method="post">
        Вид транспорта:
        <select name="transport">
            {% for t in transports %}
                <option value="{{ t }}">{{ t }}</option>
            {% endfor %}
        </select><br>
        Дата старта (ДД.ММ.ГГГГ):
        <input type="text" name="date" placeholder="01.01.2025" pattern="\d{2}\.\d{2}
\d{4}" required><br>
        Способ оплаты:
        <select name="payment">
            {% for p in payments %}
                <option value="{{ p }}">{{ p }}</option>
            {% endfor %}
        </select><br>
        <button type="submit">Подтвердить</button>
    </form>
</body>

```

admin.html {{ r.transport }} |

```

<td>{{ r.date_start }}</td>
<td>{{ r.payment_method }}</td>
<td>{{ r.review or '' }}</td>
<td>
    <form action="/admin/change_status" method="post" class="form
-status">

        <input type="hidden" name="request_id" value="{{ r.id }}"

>

        <select name="status" onchange="this.form.submit()">
            <option value="Новая" {% if r.status == 'Новая' %}sel
ected{% endif %}>Новая</option>
            <option value="Идет обучение" {% if r.status == 'Идет
обучение' %}selected{% endif %}>Идет обучение</option>
            <option value="Обучение завершено" {% if r.status ==
'Обучение завершено' %}selected{% endif %}>Завершено</option>
        </select>
    </form>
</td>
</tr>
{% endfor %}
</tbody>
</table>
</div>
</body>
</html>

```

5. Команды и полезные инструкции

5.1. Установка PostgreSQL и создание БД (если на экзамене чистая система)

```
CREATE DATABASE passRF;
```

5.2. Установка Python-зависимостей

```
pip install fastapi uvicorn psycpg2-binary python-multipart bcrypt jinja2
```

5.3. Запуск сервера

```
uvicorn main:app --reload --port 8000
```

6. Полезные фишки (готовые блоки кода для копирования)

6.1. Слайдер на Glide.js (профиль или любая страница)

6.1.1. Подключение библиотек (уже в шапке профиля)

```
<link rel="stylesheet" href="/static/glide.core.min.css">
```

```
<link rel="stylesheet" href="/static/glide.theme.min.css">
```

6.1.2. HTML-разметка слайдера (вставить в нужное место)

```
<div class="glide">
  <div class="glide__track" data-glide-el="track">
    <ul class="glide__slides">
      <li class="glide__slide"></li>
      <li class="glide__slide"></li>
      <li class="glide__slide"></li>
      <li class="glide__slide"></li>
    </ul>
  </div>
  <div class="glide__arrows" data-glide-el="controls">
    <button class="glide__arrow glide__arrow--left" data-glide-dir="<"></button>
    <button class="glide__arrow glide__arrow--right" data-glide-dir=">"></button>
  </div>
</div>
```

6.1.3. Инициализация слайдера

```
<script src="/static/glide.min.js"></script>
```

```
<script>
  new Glide('.glide', {
    type: 'carousel',
    autoplay: 3000, // автопрокрутка каждые 3 секунды
    perView: 1,
    loop: true
  }).mount();
</script>
```

6.1.4. Минимальные стили

```
.glide {
  max-width: 600px;
  margin: 20px auto;
}

.glide__slide img {
  width: 100%;
  height: 100%;
  object-fit: cover;
  display: block;
}

.glide__arrow {
  background: rgba(0,0,0,0.4);
```

```

border: none;
border-radius: 50%;
width: 40px;
height: 40px;
display: flex;
align-items: center;
justify-content: center;
}

```

6.2. Админка: пагинация, сортировка, всплывающие уведомления

6.2.1. Бэкенд (фрагмент main.py)

```

@app.get("/admin")
def get_admin(
    request: Request,
    status: str | None = Query(None),
    page: int = Query(1, ge=1),
    per_page: int = Query(5, ge=1, le=50),
    sort_by: str = Query("id", regex="^(id|date_start|status)$"),
    order: str = Query("desc", regex="^(asc|desc)$")
):
    if not request.session.get("admin"):
        return RedirectResponse("/login", status_code=302)

    # Базовый запрос с JOIN
    base = """
        SELECT requests.*, users.login, users.fio
        FROM requests, users
        WHERE requests.user_id = users.id
    """

    params = []
    if status and status != "Bce":
        base += " AND status = %s"
        params.append(status)

    # Сортировка
    sort_dir = "ASC" if order == "asc" else "DESC"
    base += f" ORDER BY {sort_by} {sort_dir}"

    # Пагинация
    offset = (page - 1) * per_page
    base += " LIMIT %s OFFSET %s"
    params.extend([per_page, offset])

    with get_db() as conn:
        cur = conn.cursor()
        cur.execute(base, params)
        requests_list = cur.fetchall()

```



```

# Подсчёт общего количества для пагинации
count_sql = "SELECT COUNT(*) FROM requests"
count_params = []
if status and status != "Bce":
    count_sql += " WHERE status = %s"
    count_params.append(status)
cur.execute(count_sql, count_params)
total = cur.fetchone()[0]

total_pages = (total + per_page - 1) // per_page

return templates.TemplateResponse("admin.html", {
    "request": request,
    "requests": requests_list,
    "cur_status": status or "Bce",
    "page": page,
    "per_page": per_page,
    "total_pages": total_pages,
    "total": total,
    "sort_by": sort_by,
    "order": order
})

@app.post("/admin/change_status")
def post_change_status(request: Request, request_id: str = Form(...), status: str = Form(...)):
    if not request.session.get("admin"):
        return RedirectResponse("/login", status_code=302)

    with get_db() as conn:
        cur = conn.cursor()
        cur.execute("UPDATE requests SET status = %s WHERE id = %s", (status, request_id))
        conn.commit()

# Редирект с параметром msg, сохраняя текущий фильтр статуса
return RedirectResponse(
    f"/admin?msg=updated&status={request.query_params.get('status', '')}&page={request.query_params.get('page', 1)}&per_page={request.query_params.get('per_page', 5)}&sort_by={request.query_params.get('sort_by', 'id')}&order={request.query_params.get('order', 'desc')}",
    status_code=302
)

```

6.2.2. Фронтенд (admin.html) – фрагменты

Заголовки таблицы с сортировкой (видимые ссылки):

```
<thead>
  <tr>
    <th><a href="?sort_by=id&order={% if sort_by == 'id' and order == 'desc' %}asc{% else %}desc{% endif %}&page={{ page }}&per_page={{ per_page }}&status={{ cur_status }}">ID</a></th>
    <th>Логин</th>
    <th>ФИО</th>
    <th>Вид транспорта</th>
    <th><a href="?sort_by=date_start&order={% if sort_by == 'date_start' and order == 'desc' %}asc{% else %}desc{% endif %}&page={{ page }}&per_page={{ per_page }}&status={{ cur_status }}">Дата старта</a></th>
    <th>Способ оплаты</th>
    <th>Отзыв</th>
    <th><a href="?sort_by=status&order={% if sort_by == 'status' and order == 'desc' %}asc{% else %}desc{% endif %}&page={{ page }}&per_page={{ per_page }}&status={{ cur_status }}">Статус</a></th>
  </tr>
</thead>
```

Пагинация после таблицы:

```
<div class="pagination">
  {% if page > 1 %}
    <a href="?page={{ page-1 }}&per_page={{ per_page }}&status={{ cur_status }}&sort_by={{ sort_by }}&order={{ order }}">← Предыдущая</a>
  {% endif %}
  Страница {{ page }} из {{ total_pages }}
  {% if page < total_pages %}
    <a href="?page={{ page+1 }}&per_page={{ per_page }}&status={{ cur_status }}&sort_by={{ sort_by }}&order={{ order }}">Следующая →</a>
  {% endif %}
</div>
```

Скрипт для alert (в конце страницы, перед </body>):

```
<script>
  if (window.location.search.includes('msg=updated')) {
    alert('Статус заявки успешно изменён!');
    // Убираем параметр msg из URL, чтобы при обновлении страницы алерт не повторялся

    let url = new URL(window.location.href);
    url.searchParams.delete('msg');
    window.history.replaceState({}, document.title, url.toString());
  }
</script>
```

```

    }
</script>
6.2.3. CSS (добавить в style.css)

```

```

.pagination {
    margin: 20px 0;
    text-align: center;
}

.pagination a {
    margin: 0 10px;
    padding: 5px 10px;
    background: #007bff;
    color: white;
    border-radius: 20px;
    text-decoration: none;
}

.pagination a:hover {
    background: #0d47a1;
}

th a {
    color: white;
    text-decoration: none;
}

th a:hover {
    text-decoration: underline;
}

```

6.3. Динамическое поле «Сдача с» (готовый код)

6.3.1. Бэкенд – добавить колонку в БД (один раз выполнить в PostgreSQL)

```
ALTER TABLE requests ADD COLUMN change_amount NUMERIC(10,2) DEFAULT NULL;
```

6.3.2. Обновлённый post_create_request

```

@app.post("/create_request")
def post_create_request(
    request: Request,
    transport: str = Form(...),
    date: str = Form(...),
    payment: str = Form(...),
    change_amount: float | None = Form(None) # новое поле
):
    user_id = request.session.get("user_id")
    if not user_id:
        return RedirectResponse("/login", status_code=302)

    if payment != "Наличные":
        change_amount = None

```

```

with get_db() as conn:
    cur = conn.cursor()
    cur.execute(
        "INSERT INTO requests (transport, date_start, payment_method, user_id, change_amount) VALUES (%s, %s, %s, %s, %s)",
        (transport, date, payment, user_id, change_amount)
    )
    conn.commit()
return HttpResponseRedirect("/profile", status_code=302)

```

6.3.3. Фронтенд (create_request.html) – добавки в форму HTML-блок (после <select name="payment">):

```

<div id="change-field-container" style="display: none;">
    <label for="change_amount">Сдача с (сумма):</label>
    <input type="number" step="0.01" name="change_amount" id="change_amount" placeholder="Например, 5000">
</div>

```

JavaScript (показывает/скрывает поле):

```

<script>
document.addEventListener('DOMContentLoaded', function() {
    const paymentSelect = document.querySelector('select[name="payment"]');
    const changeContainer = document.getElementById('change-field-container');

    function toggleChangeField() {
        if (paymentSelect && paymentSelect.value === 'Наличные') {
            changeContainer.style.display = 'block';
        } else if (changeContainer) {
            changeContainer.style.display = 'none';
        }
    }

    if (paymentSelect) {
        paymentSelect.addEventListener('change', toggleChangeField);
        toggleChangeField();
    }
});
</script>

```

6.4 Полный css (Более красивый)

```

/* Regular */
@font-face {
    font-family: 'PT Sans';
    src: url('/static/fonts/PTSans-Regular.ttf') format('truetype');
}

```

```
    font-weight: 400;
    font-style: normal;
    font-display: swap;
}

/* Bold */
@font-face {
    font-family: 'PT Sans';
    src: url('/static/fonts/PTSans-Bold.ttf') format('truetype');
    font-weight: 700;
    font-style: normal;
    font-display: swap;
}

/* Italic */
@font-face {
    font-family: 'PT Sans';
    src: url('/static/fonts/PTSans-Italic.ttf') format('truetype');
    font-weight: 400;
    font-style: italic;
    font-display: swap;
}

body {
    font-family: 'PT Sans', sans-serif;
    background-color: whitesmoke;
    align-items: center;
    text-align: center;
    margin: 0;
    padding: 10px;
    display: flex;
    flex-direction: column;
    font-size: 16px;
}

h1 {
    font-size: 36px;
    color: #007bff;
}

h2 {
    font-size: 24px;
    color: #007bff;
}
```

```

}

form {
    background-color: white;
    padding: 15px;
    box-shadow: 1px 5px 5px 1px gray;
    border-radius: 15px;
    font-weight: bold;
    max-width: 500px;
}

.elebus{
    box-shadow: 1px 5px 5px 1px gray;
    border-radius: 15px;
    max-width: 500px;
    margin-bottom: 20px;
}

input, button, textarea, select{
    padding: 10px;
    margin: 10px 0;
    border-radius: 15px;
    width: 100%;
    max-width: 400px;
    box-sizing: border-box;
}

button {
    border: none;
    background-color: #0d47a1;
    transform: 0,3s;
    cursor: pointer;
    color: white;
    width: 60%;
}

button:hover{
    background-color: rgb(40, 40, 177);
}

a{
    color: #007bff;
    font-weight: bold;
}

```

```
.nav{
  padding-bottom: 20px;
}

.card{
  background-color: white;
  padding: 15px;
  margin: 10px;
  box-shadow: 1px 5px 5px 1px gray;
  border-radius: 15px;
  width: 100%;
  max-width: 400px;
}

.form-review, .form-status{
  box-shadow: none;
  padding: 0px;
}

.form-filter{
  box-shadow: none;
  background-color: whitesmoke;
  padding-bottom: 0px;
}

.form-status select, .form-status button {
  width: auto;
}

.table-div{
  width: 100%;
  overflow-x: auto;
  margin: 10px;
}

table{
  background-color: white;
  box-shadow: 1px 5px 5px 1px gray;
  width: 100%;
  border-collapse: collapse;
}
```

```

.table-responsive {
    overflow-x: auto;
    -webkit-overflow-scrolling: touch; /* плавный скролл на iOS */
    max-width: 100%;
}

th{
    background-color: #007bff;
    color: white;
    font-weight: bold;
}

th, td{
    border: 1px solid grey;
    padding: 10px;
}

.glide {
    max-width: 600px;
}

.glide__slide img {
    width: 100%;
    height: 100%;
    object-fit: cover;
    display: block;
}

.glide__arrow {
    width: 40px;
}

@media (max-width: 480px) {
    body {
        padding: 8px;
    }
    .card, form:not(.form-filter):not(.form-status):not(.form-review) {
        margin: 12px 0;
        padding: 16px;
    }
    button[type="submit"] {
        max-width: 100%;
    }
}

```



```

th, td {
    padding: 8px 6px;
    font-size: 14px;
}
.glide {
    max-width: 100%;
}
.elebus{
    max-width: 100%;
}
}

.pagination {
    margin: 20px 0;
    text-align: center;
}
.pagination a {
    margin: 0 10px;
    padding: 5px 10px;
    background: #007bff;
    color: white;
    border-radius: 20px;
    text-decoration: none;
}
.pagination a:hover {
    background: #0d47a1;
}

/* Стили для сортируемых заголовков */
th a {
    color: white;
    text-decoration: none;
    display: inline-block;
    padding: 4px 8px;
    border-radius: 16px;
    transition: background 0.2s;
}
th a:hover {
    background: rgba(255,255,255,0.2);
    text-decoration: none;
}

/* Дополнительно: стрелка сортировки */
.sort-indicator {

```

```

    font-size: 0.8em;
    margin-left: 4px;
}

input, select, textarea, button {
    width: 100%;
    padding: 12px 16px;
    margin: 8px 0;
    border: 1px solid #ced4da;
    border-radius: 16px;
    font-size: 16px;
    box-sizing: border-box;
}

input:focus, select:focus, textarea:focus {
    border-color: #007bff;
    outline: none;
    box-shadow: 0 0 0 3px rgba(0,123,255,0.25);
}

button {
    background: linear-gradient(135deg, #007bff, #0d47a1);
    color: white;
    border: none;
    font-weight: bold;
    cursor: pointer;
    transition: all 0.2s;
}

button:hover {
    background: linear-gradient(135deg, #0d47a1, #007bff);
    transform: scale(1.02);
}

a {
    color: #007bff;
    text-decoration: none;
    font-weight: 500;
}

a:hover {
    text-decoration: underline;
}

.nav a {
    background: transparent;
    padding: 8px 16px;
    border-radius: 40px;
    transition: background 0.2s;
}

```

```
}
.nav a:hover {
    background: rgba(0,123,255,0.1);
    text-decoration: none;
}
.card {
    background: white;
    border-radius: 20px;
    box-shadow: 0 4px 12px rgba(0,0,0,0.05);
    padding: 16px;
    margin: 12px 0;
    transition: transform 0.2s, box-shadow 0.2s;
}
.card:hover {
    transform: translateY(-2px);
    box-shadow: 0 8px 24px rgba(0,0,0,0.1);
}
table {
    background: white;
    border-radius: 20px;
    overflow: hidden;
    box-shadow: 0 2px 8px rgba(0,0,0,0.05);
    width: 100%;
    border-collapse: collapse;
}
th {
    background-color: #007bff;
    color: white;
    padding: 12px;
    font-weight: 600;
}
td {
    padding: 12px;
    border-bottom: 1px solid #e9ecef;
}
tr:hover td {
    background-color: rgba(0,123,255,0.05);
}
```